LECTURE 18

A brief introduction to constraint-based verification and synthesis

There is a general approach to verification where a program is automatically converted into a logical statement of the form $\forall x.Q(x)$ such that if the statement is valid (so Q is indeed true for all x), then the program is correct. In this section we describe this process in some detail, and then in the next section we will see how synthesis can be made to interact with this process in order to generate verified programs.

Following the style of Winskell, we will be explaining the key concepts in this lecture using a simple imperative language IMP shown below.

```
expr := N | x | expr + expr | expr > expr
cmd  := x := expr
      | cmd ; cmd
      | if( expr ) cmd else cmd
      | while( expr ) cmd
      | skip
```

A program in this language manipulates a state σ that maps each variable to an integer value. Expressions evaluated under a given state produce a value, (we will use the notation $\langle expr, \sigma \rangle \rightarrow n$ to indicate that expression exp evaluates to n under state σ). And commands (often called statements) transform a state into another state (we will use the notation $\langle cmd, \sigma \rangle \rightarrow \sigma'$ to indicate that command cmd transforms state σ into state σ').

18.1 Program Logics

In order to understand this style of verification, we need to provide some background on the formalism of Program Logics, also known in the literature as Axiomatic Semantics or Hoare-Floyd style verification.

The main construction in this form of verification is the Hoare triple:

$$\{A\}cmd\{B\}$$

which indicates that for all states that satisfy some predicate A, if we execute command cmd starting from this state **and it terminates** then the resulting state will satisfy predicate B. It is important to note that this is a **partial correctness** assertion, meaning that the Hoare triple makes no claim as to whether or not

cmd actually terminates, it just says that if it does terminate, and you started from a state satisfying A, then you are guaranteed to end in a state satisfyng B.

There is a stronger version of the Hoare triple that is usually written in terms of square brackets

$$[A]cmd[B]$$

which additionally implies that cmd will terminate on all inputs that satisfy the predicate A.

Derivation Rules

Derivation rules for each language construct

$$\frac{}{\vdash \{A[x \to e]\}x := e\ \{A\}} \qquad \frac{\vdash \{A \wedge b\}c_1\ \{B\} \quad \vdash \{A \wedge not\ b\}c_2\ \{B\}}{\vdash \{A\}if\ b\ then\ c_1 else\ c_2\ \{B\}}$$

$$\frac{}{\vdash \{A\}skip\{A\}} \qquad \frac{\vdash \{A\}c_1\ \{C\} \quad \vdash \{C\}c_2\ \{B\}}{\vdash \{A\}c_1; c_2\ \{B\}} \qquad \frac{\vdash \{A \wedge b\}c\ \{A\}}{\vdash \{A\}while\ b\ do\ c\ \{A \wedge not\ b\}}$$

Can be combined together with the rule of consequence

$$\frac{\vdash A' \Rightarrow A \vdash \{A\}c\ \{B\} \vdash B \Rightarrow B'}{\vdash \{A'\}c\ \{B'\}}$$

The key idea in program logics is to provide a set of deduction rules that can be used to prove a Hoare triple. In the literature, it is common to see the notation $\vdash \{A\}cmd\{B\}$, where the $\vdash$ symbol indicates that the Hoare triple can be proven using these deductive rules. Each rule has the form:

$$\frac{Premises}{Conclusion}$$

If the premises of the rule can be proven true, then the conclusion of the rule is proven true as well. The figure on the right shows a complete set of rules for this language. There is one rule for each construct, which at first sight seem very specific. For example, the rule for assignment says that we can prove a postcondition A holds after an assignment $x := e$, if we have a precondition of the form $A[x \to e]$, which corresponds to the predicate A but with all occurences of x replaced with e. This makes sense, for example, using this rule, we can prove the following hoare triples:

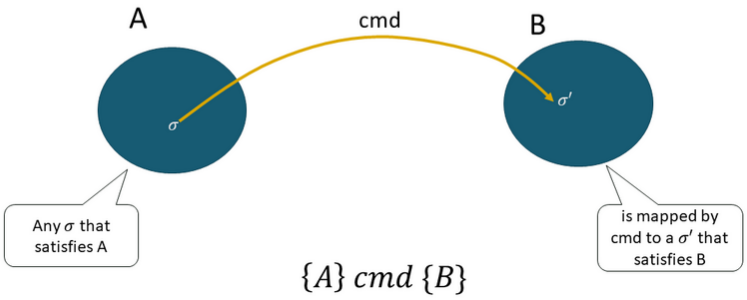
$\{y > 10\}x := y\{x > 10\}$	if $y > 10$ before x:= y then afterwards $x > 10$
$\{x + 1 > 10\}x := x + 1\{x > 10\}$	if $x + 1 > 10$ before x:= x + 1 then afterwards $x > 10$
$\{7 > t\}x := 7\{x > t\}$	if $7 > t$ before x:= 7 then afterwards $x > t$

But it also seems very restrictive, for example, suppose we want to prove

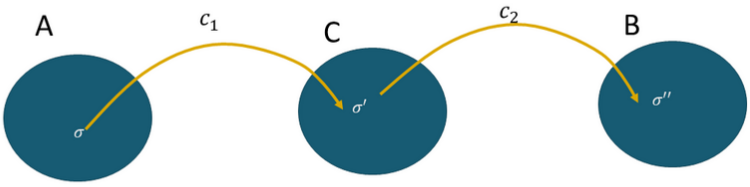
$$\{y > 7\}x := y\{x > 0\}$$

This is clearly true, but it does not quite match the form of the rule for assignment. In order to prove this, we need the rule of consequence. The rule for assignment does not allow us to directly prove the triple above, but we can use it to prove $\{y > 0\}x := y\{x > 0\}$. But because $(y > 7) \Rightarrow (y > 0)$ and $(x > 0) \Rightarrow (x > 0)$, the rule of consequence allows us to prove that the Hoare triple above is valid.

Visualizing the rules

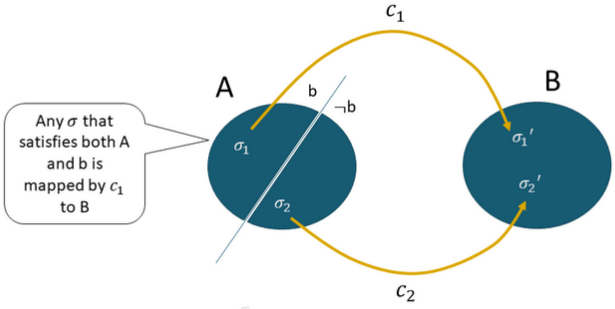


Visualizing the rules

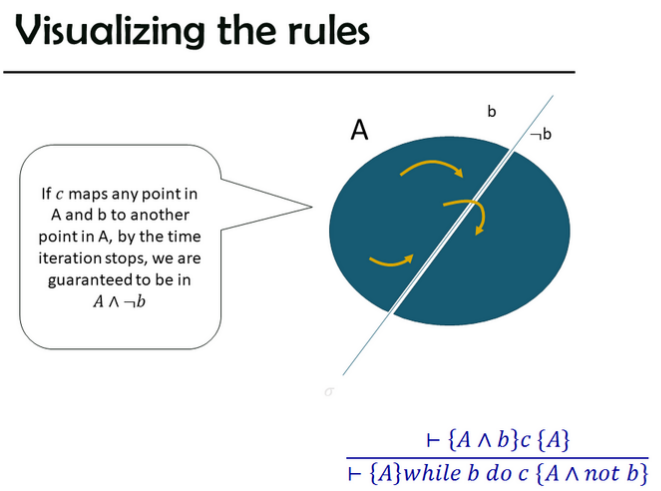


$$\frac{\vdash \{A\}c_1\ \{C\} \quad \vdash \{C\}c_2\ \{B\}}{\vdash \{A\}c_1; c_2\ \{B\}}$$

Visualizing the rules



$$\frac{\vdash \{A \wedge b\}c_1\ \{B\} \quad \vdash \{A \wedge not\ b\}c_2\ \{B\}}{\vdash \{A\}if\ b\ then\ c_1 else\ c_2\ \{B\}}$$



One way to visualize the rules is to view each predicate as a set of states satifying that predicate. So a Hoare triple $\{A\}\ cmd\ \{B\}$ ensures that any state in the set of states satifying A will be mapped by cmd to a state in the set of states satisfying B. The figure illustrates the different rules in terms of how they map states in the precondition to states in the post-condition.

For example, the rule for while loop is saying that if you can show that any state that satisfies both A and b is always mapped to another state that satisfies A, then starting from a state satisfying A, if the loop ever terminates, then when it does the state will still satisfy A, but will also satisfy $\neg b$.

18.2 Example

The following example illustrates how the rules can be used to prove a property for a given program. The figure shows a program with a pre and post condition that we wish to prove and illustrates the process of constructing the proof. Notice that every time we apply a rule, the premises of the rule create new proof obligations that need to be discharged by using other deductive rules.

Example

$$\frac{\vdash \{A \wedge b\} c_1 \{B\} \quad \vdash \{A \wedge not\ b\} c_2 \{B\}}{\vdash \{A\} if\ b\ then\ c_1 else\ c_2 \{B\}}$$

$$\frac{\begin{Bmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{Bmatrix} \quad x > y \quad \begin{array}{l} t = x - y; \\ while(t > 0) \\ \quad x = x - 1; \\ \quad y = y + 1; \\ \quad t = t - 1; \end{array} \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix} \qquad \begin{Bmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{Bmatrix} \quad not\ x > y \quad skip \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix}}{\begin{Bmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{Bmatrix} \quad \begin{array}{l} if(x > y) \\ \quad t = x - y; \\ \quad while(t > 0) \\ \quad\quad x = x - 1; \\ \quad\quad y = y + 1; \\ \quad\quad t = t - 1; \end{array} \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix}}$$

Example

$$\frac{\vdash A' \Rightarrow A \vdash \{A\} c \{B\} \vdash B \Rightarrow B'}{\vdash \{A'\} c \{B'\}}$$

$$\frac{\begin{Bmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{Bmatrix} \quad x > y \quad \begin{array}{l} t = x - y; \\ while(t > 0) \\ \quad x = x - 1; \\ \quad y = y + 1; \\ \quad t = t - 1; \end{array} \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix} \qquad \begin{Bmatrix} x = x_{old} \\ y = y_0 \\ x > y \end{Bmatrix} \text{ False } x > y \quad skip \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix}}{\begin{Bmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{Bmatrix} \quad \begin{array}{l} if(x > y) \\ \quad t = x - y; \\ \quad while(t > 0) \\ \quad\quad x = x - 1; \\ \quad\quad y = y + 1; \\ \quad\quad t = t - 1; \end{array} \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix}}$$

$$\{False\} \Rightarrow \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix} \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix} \quad skip \quad \begin{Bmatrix} y = x_{old} \\ x = y_{old} \end{Bmatrix}$$

Example

$\vdash \{A\}skip\{A\}$

$$\dfrac{\begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{pmatrix} \quad \begin{array}{l} t = x - y; \\ \text{while}(t > 0) \\ \quad x = x - 1; \\ \quad y = y + 1; \\ \quad t = t - 1; \end{array} \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix} \qquad \dfrac{\{False\} \Rightarrow \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix} \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix} \ \text{skip} \ \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}}{\begin{pmatrix} x = x_{old} \\ y = y_0 \\ x > y \end{pmatrix} \ \text{False} \ x > y \} \quad \text{skip} \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}}}{\begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{pmatrix} \quad \text{if}(x > y) \ t = x - y; \ \text{while}(t > 0) \ x = x - 1; \ y = y + 1; \ t = t - 1; \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}}$$

Example

$$\dfrac{\begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{pmatrix} \quad \begin{array}{l} t = x - y; \\ \text{while}(t > 0) \\ \quad x = x - 1; \\ \quad y = y + 1; \\ \quad t = t - 1; \end{array} \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix} \qquad \dfrac{\{False\} \Rightarrow \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix} \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix} \ \text{skip} \ \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}}{\{False\} \quad \text{skip} \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}}}{\begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \end{pmatrix} \quad \text{if}(x > y) \ t = x - y; \ \text{while}(t > 0) \ x = x - 1; \ y = y + 1; \ t = t - 1; \quad \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}}$$

Example

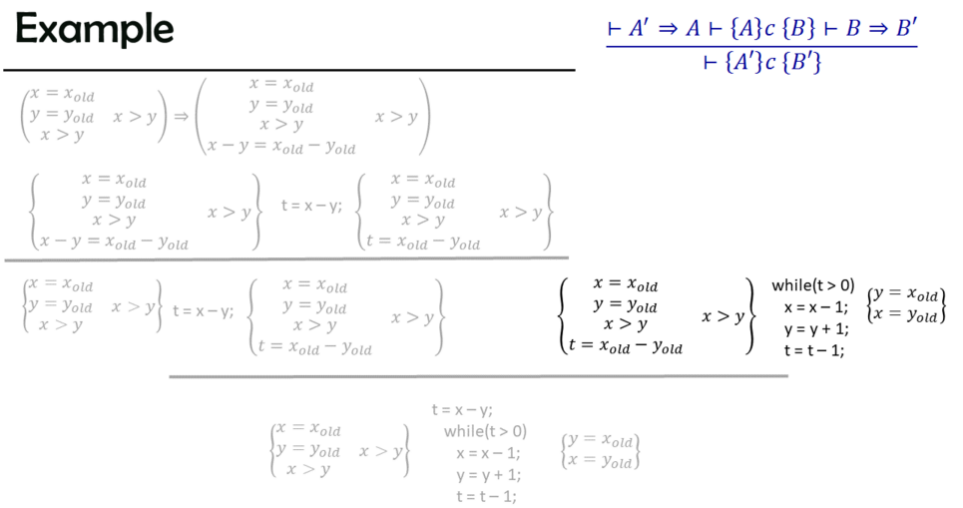
$$\frac{\vdash \{A\}c_1\{C\} \quad \vdash \{C\}c_2\{B\}}{\vdash \{A\}c_1;c_2\{B\}}$$

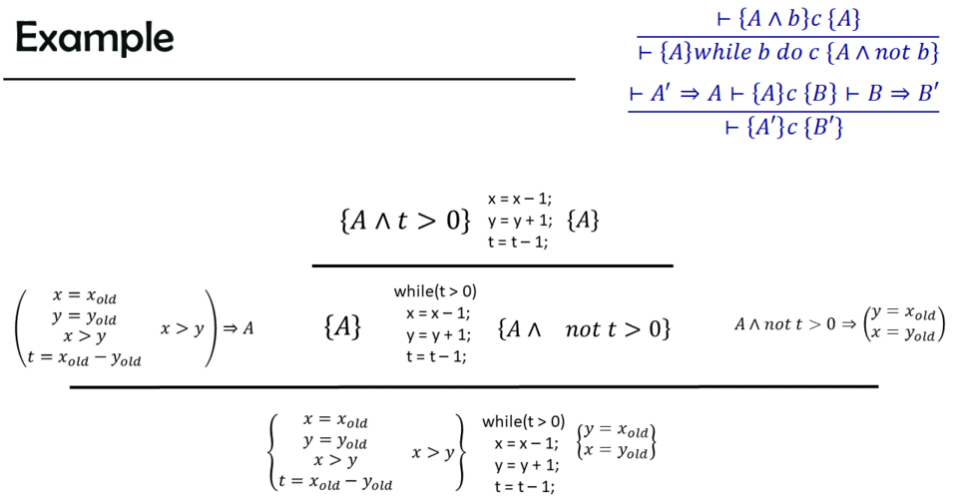
$$\frac{\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \end{matrix} \quad x > y\right\}\ \texttt{t = x - y;}\ \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{matrix} \quad x > y\right\} \qquad \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{matrix} \quad x > y\right\}\ \begin{matrix}\texttt{while(t > 0)} \\ \texttt{x = x - 1;} \\ \texttt{y = y + 1;} \\ \texttt{t = t - 1;}\end{matrix}\ \left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}}{\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \end{matrix} \quad x > y\right\}\ \begin{matrix}\texttt{t = x - y;} \\ \texttt{while(t > 0)} \\ \texttt{x = x - 1;} \\ \texttt{y = y + 1;} \\ \texttt{t = t - 1;}\end{matrix}\ \left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}}$$

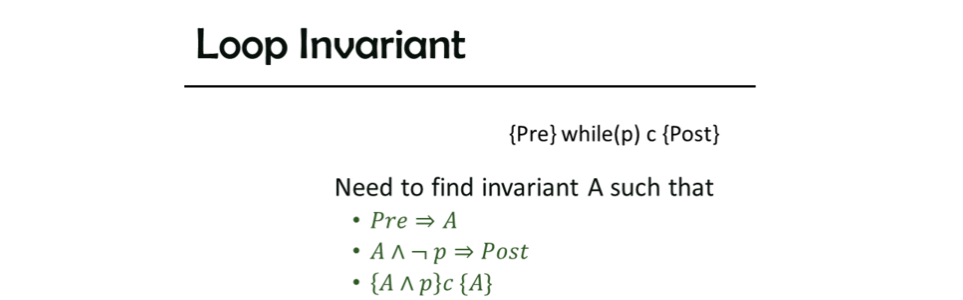
Example

$$\frac{\vdash A' \Rightarrow A \vdash \{A\}c\{B\} \vdash B \Rightarrow B'}{\vdash \{A'\}c\{B'\}}$$

$$\frac{\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \end{matrix} \quad x > y\right\} \Rightarrow \left(\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \\ x - y = x_{old} - y_{old} \end{matrix} \quad x > y\right) \qquad \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \\ x - y = x_{old} - y_{old} \end{matrix} \quad x > y\right\}\ \texttt{t = x - y;}\ \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{matrix} \quad x > y\right\}}{\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \end{matrix} \quad x > y\right\}\ \texttt{t = x - y;}\ \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{matrix} \quad x > y\right\} \qquad \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{matrix} \quad x > y\right\}\ \begin{matrix}\texttt{while(t > 0)} \\ \texttt{x = x - 1;} \\ \texttt{y = y + 1;} \\ \texttt{t = t - 1;}\end{matrix}\ \left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}}{\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \end{matrix} \quad x > y\right\}\ \begin{matrix}\texttt{t = x - y;} \\ \texttt{while(t > 0)} \\ \texttt{x = x - 1;} \\ \texttt{y = y + 1;} \\ \texttt{t = t - 1;}\end{matrix}\ \left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}}$$







The figure illustrates the first few steps of the proof. The proof proceeds very mechanically until we get to a proof obligation involving loops. This proof

obligation is problematic because the rule for loops has a very particular form. It is given in terms of a predicate A that must be preserved by each iteration of the loop. However, the pre and post-conditions of the while loop in the example do not match this form. This means that in order to make the proof work, we need to discover a predicate A that is then connected to the pre and post conditions using the rule of consequence. From the figure, we can see that the use of the rule of consequence together with the rule for loops leaves us with three proof obligations.

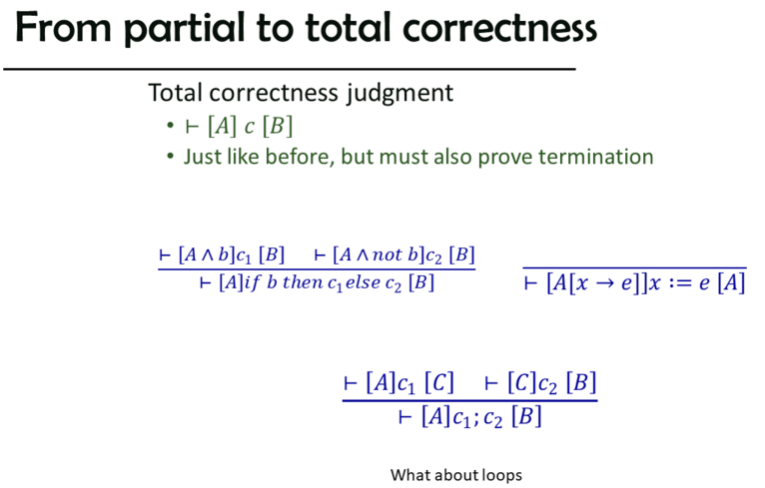
- The precondition must imply A.
- A and the negation of the loop condition must imply the postcondition.
- When the loop condition is true, A must be preserved by the loop body.

This predicate A is a loop invariant, and in general, it cannot be discovered algorithmically. In the case of the example, we can see that the conditions can be satisfied by the predicate below.

$$\begin{array}{rcll} & & x; = y_{old} + t \\ A & \equiv & \wedge & y = x_{old} - t \\ & & \wedge & t >= 0 \end{array}$$

Using this invariant, we can now complete the proof that the Hoare triple in the example is indeed correct.

18.3 From partial to total correctness



Rank function

Function F of the state that

- a) Maps state to an integer
- b) Decreases with every iteration of the loop
- c) Is guaranteed to stay greater than zero
- Also called variant function

z is just a fresh variable not used elsewhere in the expressions

$$\frac{\vdash [A \wedge b \wedge F = z]c\,[A \wedge F < z] \quad \vdash A \wedge b \Rightarrow F \geq 0}{\vdash [A] while\ b\ do\ c\ [A \wedge not\ b]}$$

Example

Can we prove this?

```
[ x=x0 and y=y0 and x > y ]
if(x > y){
 t = x − y;
 while(t > 0){
  x = x − 1;
  y = y + 1;
  t = t − 1;
 }
}
[y=x0 and x=y0 ]
```

Example

$$\frac{\vdash [A \wedge b \wedge F = z]c\,[A \wedge F < z] \quad \vdash A \wedge b \Rightarrow F \geq 0}{\vdash [A] while\ b\ do\ c\ [A \wedge not\ b]}$$

$$A \equiv \begin{array}{l} x = y_{old} + t \\ \wedge\ y = x_{old} - t \\ \wedge\ t \geq 0 \end{array} \qquad F \equiv t$$

$$[A \wedge t > 0 \wedge t = z] \quad \begin{array}{l} x = x - 1; \\ y = y + 1; \\ t = t - 1; \end{array} \quad [A \wedge (t < z)] \qquad A \wedge t > 0 \Rightarrow t \geq 0 \ \checkmark$$

$$[A] \quad \begin{array}{l} \text{while(t > 0)} \\ \quad x = x - 1; \\ \quad y = y + 1; \\ \quad t = t - 1; \end{array} \quad [A \wedge \ \ not\ t > 0]$$

Example

$$\frac{\vdash [A \wedge b \wedge F = z]c\,[A \wedge F < z] \quad \vdash A \wedge b \Rightarrow F \geq 0}{\vdash [A]\mathit{while}\ b\ \mathit{do}\ c\ [A \wedge \mathit{not}\ b]}$$

$$\frac{\dfrac{\left[\begin{smallmatrix} x = y_{old} + t \\ \wedge\ y = x_{old} + t \\ \wedge\ t \geq 0 \end{smallmatrix}\right] \texttt{x = x - 1; y = y + 1;} \left[\begin{smallmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \\ \wedge\ (t-1) \geq 0 \end{smallmatrix} \wedge (t = z)\right] \qquad \left[\begin{smallmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \\ \wedge\ (t-1) \geq 0 \end{smallmatrix} \wedge (t = z)\right] \texttt{t = t - 1;} \left[\begin{smallmatrix} x = y_{old} + t \\ \wedge\ y = x_{old} - t \\ \wedge\ t \geq 0 \end{smallmatrix} \wedge (t < z)\right]}{[A \wedge t > 0 \wedge t = z]\ \texttt{x = x - 1; y = y + 1; t = t - 1;}\ [A \wedge (t < z)]} \qquad A \wedge t > 0 \Rightarrow t \geq 0}{[A]\ \texttt{while(t > 0) x = x - 1; y = y + 1; t = t - 1;}\ [A \wedge \ \mathit{not}\ t > 0]}$$

Example

$$\frac{}{\vdash [A[x \to e]]x := e\,[A]}$$

$$\frac{\dfrac{\left(\begin{smallmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \\ \wedge\ (t-1) \geq 0 \end{smallmatrix} \wedge (t = z)\right) \Rightarrow \left(\begin{smallmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \\ \wedge\ (t-1) \geq 0 \end{smallmatrix} \wedge (t-1 < z)\right) \qquad \left[\begin{smallmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \\ \wedge\ (t-1) \geq 0 \end{smallmatrix} \wedge (t-1 < z)\right] \texttt{t = t - 1;} \left[\begin{smallmatrix} x = y_{old} + t \\ \wedge\ y = x_{old} - t \\ \wedge\ t \geq 0 \end{smallmatrix} \wedge (t < z)\right]}{\dots \qquad \left[\begin{smallmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \\ \wedge\ (t-1) \geq 0 \end{smallmatrix} \wedge (t = z)\right] \texttt{t = t - 1;} \left[\begin{smallmatrix} x = y_{old} + t \\ \wedge\ y = x_{old} - t \\ \wedge\ t \geq 0 \end{smallmatrix} \wedge (t < z)\right]}{[A \wedge t > 0 \wedge t = z]\ \texttt{x = x - 1; y = y + 1; t = t - 1;}\ [A \wedge (t < z)]} \qquad A \wedge t > 0 \Rightarrow t \geq 0}{[A]\ \texttt{while(t > 0) x = x - 1; y = y + 1; t = t - 1;}\ [A \wedge \ \mathit{not}\ t > 0]}$$

Proving total correctness is very similar. In fact, the total correctness rules for everything that doesn't involve a loop are exactly the same. It is only with loops that termination becomes an issue. The rule for loops is more involved, because in addition to the invariant A, it also includes a *Rank function* F, also refered in the literature as a *Variant function*. The Rank function allows us to construct an inductive argument for the termination of the loop. The Rank function maps the state to an integer value and imposes two conditions:

- It is greater than or equal to zero for any state where the invariant and the loop condition hold.
- It decreases with every iteration of the loop

For the running example above, the variant function is just t. It is easy to prove that that the loop body will decrement t on every iteration, and that $t \geq 0$ for every state that satisfies the loop condition. This implies that eventually the loop condition will have fail.

18.4 Weakest Preconditions and Verification Conditions

Weakest Preconditions

$$P = wpc(c, A)$$

Command — Predicate

Weakest predicate P such that $\{P\}\ c\ \{A\}$

- P weaker than Q iff $Q \Rightarrow P$



$wpc(skip, Q) = Q$

$wpc(x := e, Q) = Q[x \rightarrow e]$

$wpc(C_1; C_2, Q) = wpc(C_1, wpc(C2, Q))$

$wpc(if\ B\ then\ C_1\ else\ C_2, Q) = (B \wedge wpc(C_1, Q)) \vee (\neg B \wedge wpc(C_2, Q))$

Weakest Precondition

While-loop is tricky

- Let $W = wpc(while\ e\ do\ c, B)$
- then,

$$W = e \Rightarrow wpc(c, W) \ \wedge \ \neg e \Rightarrow B$$

Verification Condition

Stronger than the weakest precondition

Can be computed by using an invariant

$$VC(while_I\ e\ do\ c, B) = I \wedge \ \forall x_1, \ldots x_n\ I \Rightarrow (e \Rightarrow VC(c, I) \wedge \neg e \Rightarrow B)$$

- Where x_i are variables modified in c.

The approach outlined above allows us to prove properties of complex programs, but it is not very mechanical, as it involves a fair amount of human judgement, not just in determining the loop invariants and rank functions, but also in determining when and how to apply the rule of consequence. We will now introduce an approach that is more mechanical, although it is still going to require users to provide loop invariants.

To explain this approach, first we need to introduce the concept of a *weakest precondition*. The weakest precondition $wpc(cmd, B)$ is a precondition such that the Hoare triple $\{wpc(cmd, B)\}cmd\{B\}$ is valid and any other valid precondition A is stronger, i.e. $\vdash \{A\}cmd\{B\}$ valid iff $A \Rightarrow wpc(cmd, B)$.

Now, suppose we had an algorithmic way of computing $wpc(cmd, B)$ for any command cmd and any postcondition B. Then checking whether a Hoare triple $\{A\}cmd\{B\}$ is valid or not would reduce to checking whether the implication $A \Rightarrow wpc(cmd, B)$ holds. For loop free programs, it is possible and relatively easy to compute wpc. The rules for computing it are shown in the figure.

As one might expect, though, it is in general not possible to compute wpc algorithmically for programs with loops. Instead, we are going to settle for a *Verification Condition* $vc(cmd, B)$. The verification condition $vc(cmd, B)$ is a valid precondition, so $\{vc(cmd, B)\}cmd\{B\}$ is guaranteed to be valid. However, it is not guaranteed to be the weakest precondition. This means that if $A \Rightarrow vc(cmd, B)$, then we can be sure that $\vdash \{A\}cmd\{B\}$, just from the rule of consequence. However, if $A \Rightarrow vc(cmd, B)$ does not hold, then we have no way of knowing whether it was because the Hoare triple $\{A\}cmd\{B\}$ is not valid, or whether the triple is valid, but $vc(cmd, B)$ is just not good enough to prove it.

The rules for computing the vc for constructs other than loops are the same as those for computing the wpc. For loops, the rule is shown in the figure. Note that the rule is given in terms of a loop invariant I, that we expect to annotate the loop. It is interesting to note that the property that $\vdash \{vc(cmd, B)\}cmd\{B\}$ will hold regardless of what invariant we provide. We could even use a random expression generator to generate an invariant, and the property would still hold. This means that we do not have to trust whoever provides the loop invariant, as long as $A \Rightarrow vc(cmd, B)$, then we will know that $\vdash \{A\}cmd\{B\}$. The catch, of course, is that if we provide a garbage invariant, $vc(cmd, B)$ is likely to evaluate to $false$. It turns out $false$ is a valid precondition for any postcondition, just not a very useful one when it comes to proving things.